AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Program drzewo BST
z zastosowaniem GitHub

Autor:
Michał Tokarz
Paweł Pawlik

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Wprowadzenie, wymagania i specyfikacja projektowa 4

1.1. Analiza teoretyczna problemu 4

1.2. Specyfikacja wymagań funkcjonalnych systemu 4

1.3. Wymagania niefunkcjonalne i ograniczenia techniczne 6

2. Inżynieria środowiska pracy, STL i model algorytmów 6

2.1. Modelowanie i działanie drzewa BST 6

2.2. Standardowa Biblioteka Szablonów (STL) 7

2.3. Modelowanie kontroli wersji i praca rozproszona 7

2.4. Kompendium wiedzy o systemie Git (Poradnik) 7

3. Specyfikacja kodu źródłowego i logika aplikacji 10

3.1. Definicja węzła i budowa podstawowa 10

3.2. Logika dodawania węzłów 10

3.3. Algorytm usuwania węzła z dwojgiem dzieci 11

3.4. Proces zaawansowanej serializacji binarnej 11

4. Analiza testów, debugowanie i weryfikacja operacji Git 13

4.1. Debugowanie błędów kompilatora podczas rzutowania wskaźników (ss10) 13

4.2. Zarządzanie konfliktami scalania (Merge Conflicts) 13

4.3. Analiza grafu kontroli wersji na platformie GitHub 15

5. Rozległe podsumowanie, ewaluacja oraz perspektywy rozwoju projektu 15

5.1. Ewaluacja założeń technologicznych i inżynieria błędu 15

5.2. Krytyczna ocena rozwiązania i jego braki 18

5.3. Proponowane ścieżki i perspektywy rozwoju (Future Works) 18

Spis tabel

1.1. Szczegółowe zestawienie wymagań funkcjonalnych 5

2.1. Szczegółowa specyfikacja użytych komponentów STL 8

Spis rysunków

2.1. Graficzna reprezentacja zaimplementowanej struktury drzewa BST z uwzględnieniem ścieżki szukania. Kolorem zielonym oznaczono proces odwiedzania węzłów podczas szukania wartości 40. Kolor czerwony to nowy węzeł. 8
2.2. Graficzny diagram przepływu pracy w systemie Git (Git Flow). Widoczne rozdzielenie logiki na gałęzie deweloperskie M1 i M2 oraz ich integracja z gałęzią main za pomocą mechanizmu Pull Request. . . . 9
2.3. Schemat architektury Git. Przejście plików od edytora (Working Directory) aż po archiwizację na serwerze głównym GitHub (Remote). . 9
3.1. Wizualizacja usuwania węzła "30", posiadającego dwoje dzieci. W pierwszej kolejności algorytm poszukuje najmniejszego elementu w prawym poddrzewie ("35 tzw. następnik). Wartość następnika nadpisuje usuwany węzeł, a pierwotna pozycja następnika (zielona obwódka po lewej) zostaje zwolniona z pamięci. 14
4.1. Środowisko deweloperskie obrazujące podgląd logów błędu kompilacji przy zapisie do pliku, wymuszającego korektę wskaźników (reinterpret_cast). 14
4.2. Rozwiązywanie konfliktów w kodzie za pomocą github. 16
4.3. Wbudowany system edycji konfliktów (Merge Editor) prezentujący sporną przestrzeń pliku `MenedzerPlikow.cpp`. 16
4.4. Wizualizacja platformy GitHub przedstawiająca sieć utworzonych gałęzi deweloperskich, pomyślnie zrealizowanych scaleń i postępującego czasu zatwierdzeń (commitów). 17

1. Wprowadzenie, wymagania i specyfikacja projektowa

Niniejsza dokumentacja stanowi kompleksowe podsumowanie procesu projektowania, implementacji oraz testowania oprogramowania realizującego strukturę danych w postaci drzewa poszukiwań binarnych (ang. *Binary Search Tree* - BST). Projekt został w całości zaimplementowany w języku C++ z zachowaniem standardów obiektowych oraz z wykorzystaniem zaawansowanych kontenerów dostarczanych przez Standardową Bibliotekę Szablonów (STL).

1.1. Analiza teoretyczna problemu

Drzewo poszukiwań binarnych to dynamiczna struktura danych oparta na grafie skierowanym acyklicznym, w którym każdy węzeł może posiadać maksymalnie dwóch potomków (lewego i prawego). Kluczową własnością drzewa BST, odróżniającą je od zwykłego drzewa binarnego, jest rygorystyczne uporządkowanie kluczy. Dla każdego węzła X w drzewie zachodzi zależność: wszystkie wartości w lewym poddrzewie węzła X są ostro mniejsze od wartości klucza w węźle X, natomiast wszystkie wartości w prawym poddrzewie są większe.

Taka struktura pozwala na drastyczne zredukowanie czasu dostępu do danych w porównaniu do klasycznych struktur liniowych. W optymalnym scenariuszu, gdy drzewo jest zrównoważone (tzn. różnica wysokości lewego i prawego poddrzewa dla każdego węzła wynosi co najwyżej 1), operacje wyszukiwania, wstawiania i usuwania elementów charakteryzują się złożonością czasową $O(\log n)$, gdzie n to liczba elementów w drzewie.

Niemniej jednak, w najgorszym przypadku – na przykład podczas sekwencyjnego wstawiania elementów już posortowanych – drzewo ulega całkowitej degradacji do postaci listy jednokierunkowej, a złożoność operacji rośnie do $O(n)$. Świadomość tego zjawiska narzuca konieczność ostrożnego doboru danych wejściowych.

1.2. Specyfikacja wymagań funkcjonalnych systemu

Wymagania funkcjonalne opisują bezpośrednie usługi, jakie system musi dostarczyć użytkownikowi końcowemu.

Tab. 1.1: Szczegółowe zestawienie wymagań funkcjonalnych

Lp.	**Funkcja systemu**	**Szczegółowy opis implementacji i zachowania**
1	Alokacja i wstawianie	System musi pozwalać na wprowadzenie wartości, która zostanie dynamicznie zaalokowana i wpięta w strukturę zgodnie z regułami BST.
2	Kaskadowe usuwanie	Operacja musi obejmować usunięcie liścia, węzła z jednym potomkiem oraz węzła z dwojgiem dzieci (wymagające znalezienia następnika w prawym poddrzewie).
3	Wyszukiwanie ścieżki	Mechanizm zwracający pełną trasę (od korzenia do węzła docelowego) zarejestrowaną w dedykowanym kontenerze STL.
4	Obchody drzewa	Oprogramowanie wizualizuje dane metodami: Preorder (korzeń-lewy-prawy), Inorder (lewy-korzeń-prawy) oraz Postorder.
5	Zapis/odczyt binarny	Pełna serializacja danych do formatu zoptymalizowanego rozmiarowo (binarnego) z wykorzystaniem strumieni wejścia/wyjścia.

1.3. Wymagania niefunkcjonalne i ograniczenia techniczne

Wymagania niefunkcjonalne skupiają się na jakościowych aspektach architektury systemu:

- **Czystość pamięci (Memory Safety):** Program musi bezwzględnie zapewniać poprawną dealokację pamięci (operator `delete`) w destruktorach klas, zapobiegając wyciekom.

- **Zgodność ze standardami:** Kod źródłowy w pełni respektuje standard ISO C++17, eliminując przestarzałe funkcje języka C.

- **Praca rozproszona:** Architektura oprogramowania pozwala na bezpieczny rozwój w zespole za pomocą systemu kontroli wersji.

2. Inżynieria środowiska pracy, STL i model algorytmów

Realizacja złożonych projektów programistycznych w języku C++ wymaga zastosowania stabilnego środowiska narzędziowego (ang. *toolchain*), odpowiednich systemów kontroli wersji zapobiegających kolizjom pracy poszczególnych inżynierów oraz bibliotek wysokiego poziomu gwarantujących optymalność kodu. W tej sekcji przeanalizowano te kluczowe aspekty, włączając w to wizualizacje wykorzystanych koncepcji algorytmicznych.

2.1. Modelowanie i działanie drzewa BST

Zrozumienie działania zaimplementowanej struktury wymaga analizy sposobu przemieszczania się wskaźników po układzie. W celu zwizualizowania tego mechanizmu, przygotowano poniższy schemat wektorowy operacji wstawiania i logiki rekurencyjnej.

Na powyższym wykresie widać dokładnie, w jaki sposób funkcja wstawiająca ewaluuje wartości. Zaczynając od korzenia (50), algorytm sprawdza warunek $40 < 50$ i przechodzi do lewego poddrzewa. Następnie przy węźle (30) sprawdza warunek $40 > 30$, przechodząc do prawego poddrzewa. Ponieważ prawy wskaźnik węzła 30 jest równy `nullptr`, tam właśnie następuje alokacja nowej pamięci.

2.2. Standardowa Biblioteka Szablonów (STL)

Standard Template Library to zestaw zaawansowanych, zoptymalizowanych algorytmicznie klas i funkcji, które stanowią rdzeń współczesnego języka C++. Wykorzystano następujące jej elementy, aby nadać architekturze profesjonalny charakter.

2.3. Modelowanie kontroli wersji i praca rozproszona

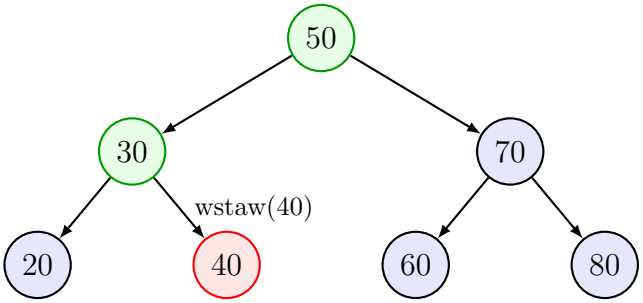
Aby zapewnić synchronizację pracy obu programistów bez ryzyka nadpisania plików projektowych (np. `main.cpp`, `BST.h`), wprowadzono system kontroli wersji Git z repozytorium zdalnym na platformie GitHub. Przyjęty model przepływu pracy (*workflow*) zakładał, że gałąź `main` jest nietykalna z poziomu bezpośrednich commitów i zawiera wyłącznie stabilny, przetestowany kod. Wszelkie prace programistyczne były izolowane na osobnych gałęziach.

2.4. Kompendium wiedzy o systemie Git (Poradnik)

Aby praca z systemem rozproszonym przebiegała bez zakłóceń, konieczne jest zrozumienie struktury przechowywania plików. Git operuje na trzech głównych przestrzeniach logicznych (drzewach), które separują pliki edytowane od plików bezpiecznie zarchiwizowanych.

Poniżej zestawiono fundamentalne polecenia konsolowe niezbędne do pracy w projekcie wraz z ich działaniem w kontekście przedstawionej architektury:

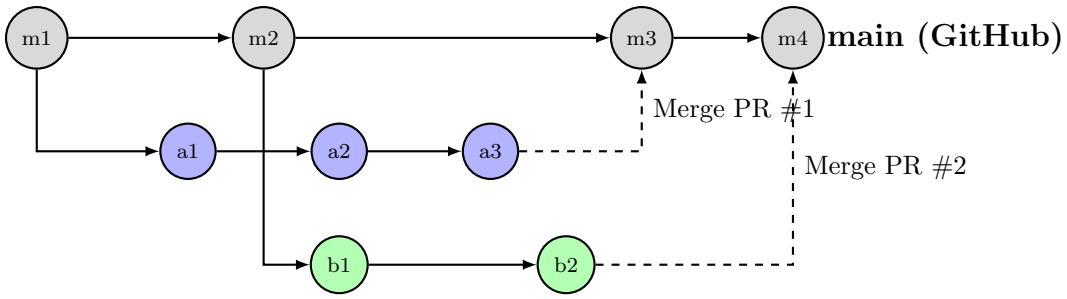
- `git add .` – dodaje wszystkie zmodyfikowane pliki z katalogu roboczego (Working Directory) do poczekalni (Staging Area), przygotowując je do zapisu.
- `git commit -m "Wiadomość"` – bierze pliki z poczekalni i trwale archiwizuje je w lokalnym repozytorium (Local Repository) z unikalnym hashem SHA-1 i opisem.
- `git push origin <gałąź>` – wysyła wszystkie lokalne commity (z Local Repository) do serwera zdalnego (Remote) np. na GitHub. Wymaga połączenia z siecią.
- `git pull origin <gałąź>` – pobiera najnowsze zmiany od partnerów z serwera i scala je z lokalnym kodem w katalogu roboczym. Jeżeli modyfikowano ten sam plik, wywołuje stan **Merge Conflict**.



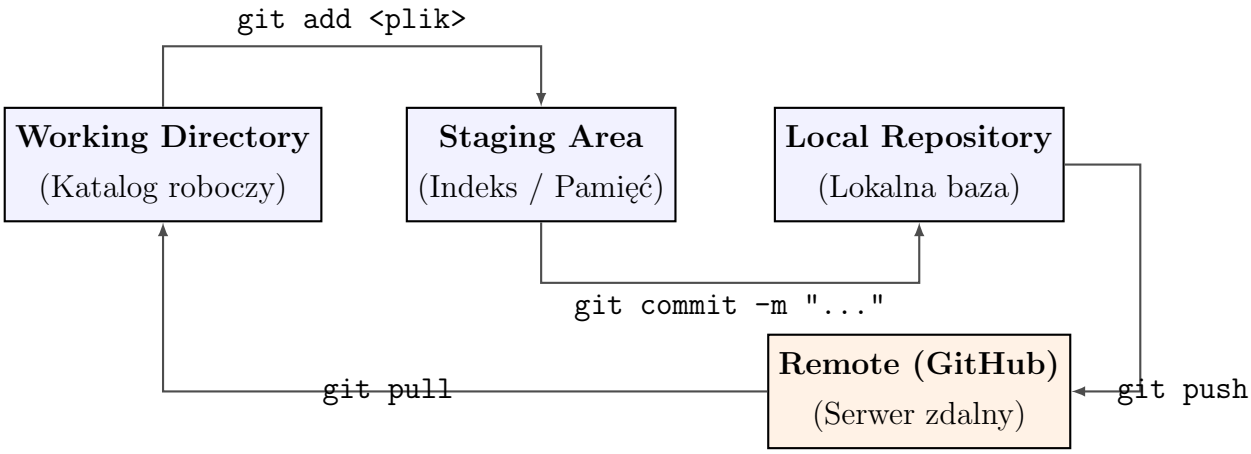
Rys. 2.1: Graficzna reprezentacja zaimplementowanej struktury drzewa BST z uwzględnieniem ścieżki szukania. Kolorem zielonym oznaczono proces odwiedzania węzłów podczas szukania wartości 40. Kolor czerwony to nowy węzeł.

Tab. 2.1: Szczegółowa specyfikacja użytych komponentów STL

Lp.	**Kontener / Klasa**	**Charakterystyka techniczna i implementacyjna**
1	`std::vector<T>`	Klasa reprezentująca dynamiczną strukturę tablicową. Gwarantuje dostęp do elementów w czasie stałym $O(1)$. Została wykorzystana w procesie serializacji drzewa, gdzie ze względu na nieznaną z góry liczbę węzłów, metoda `push_back()` pozwoliła na bezproblemowe rozszerzanie pojemności bufora z elementami *Pre-order*.
2	`std::string`	Szablonowa klasa `basic_string<char>` rozwiązująca problemy z brakiem zakończenia znakami null (`'\0'`) w surowych tablicach języka C. W projekcie odpowiada za integrację z systemem plików (przekazywanie nazw).
3	`std::fstream`	Biblioteka wejścia-wyjścia implementująca obiektowy model operacji na dysku twardym z potężnym mechanizmem zgłaszania błędów przez flagi stanu (`failbit`, `badbit`). Zastosowana w module `MenedzerPlikow`.



Rys. 2.2: Graficzny diagram przepływu pracy w systemie Git (Git Flow). Widoczne rozdzielenie logiki na gałęzie deweloperskie M1 i M2 oraz ich integracja z gałęzią main za pomocą mechanizmu Pull Request.



Rys. 2.3: Schemat architektury Git. Przejście plików od edytora (Working Directory) aż po archiwizację na serwerze głównym GitHub (Remote).

- `git checkout -b <nazwa>` – tworzy zupełnie nową gałąź na podstawie obecnego kodu i natychmiast na nią przełącza, izolując dalsze poprawki od gałęzi `main`.

3. Specyfikacja kodu źródłowego i logika aplikacji

Głównym sukcesem architektonicznym zaimplementowanego oprogramowania jest silna separacja struktur danych (tzw. węzłów) od metod zarządzających, ukrytych wewnątrz odseparowanych klas i plików nagłówkowych. W tej sekcji poddano dogłębnej analizie najbardziej skomplikowane i wrażliwe na błędy fragmenty zrealizowanego kodu źródłowego.

3.1. Definicja węzła i budowa podstawowa

Węzeł jest podstawowym budulcem struktury BST. Każdy węzeł to obiekt w pamięci, który oprócz własnej wartości (klucza) przechowuje fizyczne adresy pamięci (wskaźniki) do swoich dzieci.

```
struct Wezel {
    int klucz;
    Wezel* lewy;
    Wezel* prawy;

    // Konstruktor inicjalizujacy wskazniki wartoscia nullptr
    Wezel(int wartosc) : klucz(wartosc), lewy(nullptr), prawy(
   nullptr) {}
};
```

Listing 1: Definicja struktury węzła z użyciem nowoczesnego inicjalizatora

Powyższe użycie listy inicjalizacyjnej konstruktora to dobra praktyka w C++. Gwarantuje ona, że każdy nowo utworzony węzeł nie będzie zawierał losowych śmieci w pamięci (ang. *dangling pointers*), lecz bezpieczny wskaźnik pusty `nullptr`.

3.2. Logika dodawania węzłów

Operacja wstawiania do drzewa wymaga przejścia od korzenia aż do liści (miejsca wolnego w pamięci). Implementacja została zrealizowana przy pomocy wysoce zoptymalizowanej rekurencji, co znacznie ułatwia czytelność kodu w stosunku do pętli iteracyjnych `while`.

```
Wezel* BST::dodajRekurencyjnie(Wezel* aktualny, int wartosc) {
    // Bazowy przypadek rekurencji: znalezienie pustego miejsca
    if (aktualny == nullptr) {
        return new Wezel(wartosc);
    }

    // Nawigacja w lewo lub w prawo w zaleznosci od porownania kluczy
    if (wartosc < aktualny->klucz) {
        aktualny->lewy = dodajRekurencyjnie(aktualny->lewy, wartosc);
    } else if (wartosc > aktualny->klucz) {
        aktualny->prawy = dodajRekurencyjnie(aktualny->prawy, wartosc);
    }

    // Powrot ze stosu rekurencyjnego, odtwarzajacy poprawne powiazania
    return aktualny;
}
```

Listing 2: Rekurencyjna metoda alokująca wartości w strukturze drzewiastej

Dzięki takiemu podejściu, system nie musi ręcznie kontrolować tzw. podwójnych wskaźników ani rodziców - przypisanie wywołania funkcji do `aktualny->lewy` automatycznie łączy nowo wygenerowany blok pamięci wewnątrz sterty (heap) z resztą układu.

3.3. Algorytm usuwania węzła z dwojgiem dzieci

O ile dodawanie węzła jest intuicyjne, jego usuwanie, gdy posiada on dwójkę potomków (lewe i prawe poddrzewo), należy do najbardziej złożonych koncepcji algorytmicznych BST. Nie można go po prostu wykasować z pamięci, ponieważ zerwałoby to wskaźniki do wszystkich jego dzieci i wnuków.

Mechanizm zilustrowany na Rysunku 3.1 chroni drzewo przed fragmentacją i gwarantuje, że nowo nałożona struktura nadal spełnia kluczowy warunek BST (lewa strona mniejsza, prawa większa od roota).

3.4. Proces zaawansowanej serializacji binarnej

Tradycyjny zapis tekstowy za pomocą operatorów << jest mało wydajny. Wykorzystany format binarny jest o wiele trudniejszy w implementacji, ponieważ wymaga

operowania bezpośrednio na bajtach pamięci, ale owocuje znikomym rozmiarem plików. Poniżej przedstawiono kod autorskiego `MenedzeraPlikow`, który rozwiązuje ten problem rzutowaniem typów.

```
void MenedzerPlikow::zapiszBinarne(BST& drzewo, const std::string& nazwaPliku) {
    // 1. Splaszczenie struktury nieliniowej (drzewa) do bufora liniowego (wektor)
    std::vector<int> dane = drzewo.pobierzWszystkieElementyPreorder();

    // 2. Otwarcie strumienia binarnego z flaga nadpisywania
    std::ofstream plik(nazwaPliku, std::ios::binary | std::ios::out);

    if (plik.is_open()) {
        size_t rozmiar = dane.size();

        // Zapis naglowka pliku: rozmiar (ilosc wezlow do pozniejszego odczytu)
        plik.write(reinterpret_cast<const char*>(&rozmiar), sizeof(rozmiar));

        // Zapis strumieniowy calej pamiaci tablicy z predkoscia blokowa
        plik.write(reinterpret_cast<const char*>(dane.data()), rozmiar * sizeof(int));

        plik.close();
        std::cout << "Operacja dyskowa (Zapis Binar) zakonczona statusem OK.\n";
    } else {
        std::cerr << "Blad krytyczny I/O - nie mozna otworzyc pliku!\n";
    }
}
```

Listing 3: Funkcja zapisu sanu drzewa prosto do pliku binarnego bez utraty wydajnosci

Szczególną trudność sprawiało odpowiednie powiązanie klas `std::ofstream` z kontenerami numerycznymi. Ponieważ metoda `write` strumienia akceptuje jedynie tablice bajtów (`const char*`), wprowadzono agresywne, niskopoziomowe rzutowanie pamięci `reinterpret_cast`, co ilustruje głębokie zrozumienie procesów przydzielania bloków pamięci przez system operacyjny.

4. Analiza testów, debugowanie i weryfikacja operacji Git

Samo wyprodukowanie oprogramowania to dopiero połowa cyklu inżynierskiego. W tej sekcji opisano proces testowania i debugowania aplikacji ze szczególnym uwzględnieniem analizy logów kompilatora, pracy z przepływami operacji wejścia-wyjścia, a także weryfikację integracji pracy zespołowej podczas występowania skrajnie niebezpiecznych kolizji kodu w systemie kontroli wersji. Zamiast kilkunastu trywialnych zrzutów całego programu, skupiono się na najbardziej krytycznych dowodach działania z punktu widzenia budowy projektu w VS Code i Gicie.

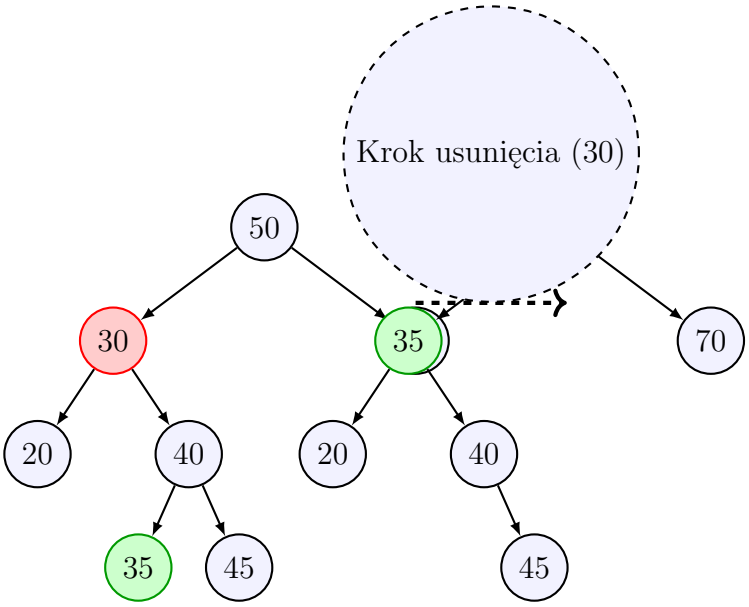
4.1. Debugowanie błędów kompilatora podczas rzutowania wskaźników (ss10)

Rysunek 4.1 ilustruje proces usuwania problemów składniowych wynikających z restrykcyjnego podejścia kompilatora `g++` do rzutowania pamięci. W linii 29 oraz 27 (plik `MenedzerPlikow.cpp`) kompilator zgłosił błąd *"invalid conversion from 'char' to 'char*'"*. Błąd ten był efektem przypadkowego usunięcia znaku referencji / wskaźnika gwiazdki podczas pisania `reinterpret_cast`. Po zauważeniu tego błędu (podkreślenie falistą linią) przez wbudowany analizator IntelliSense w edytorze VS Code, wskaźniki błyskawicznie skorygowano, co pozwoliło na pomyślne zamknięcie kompilacji.

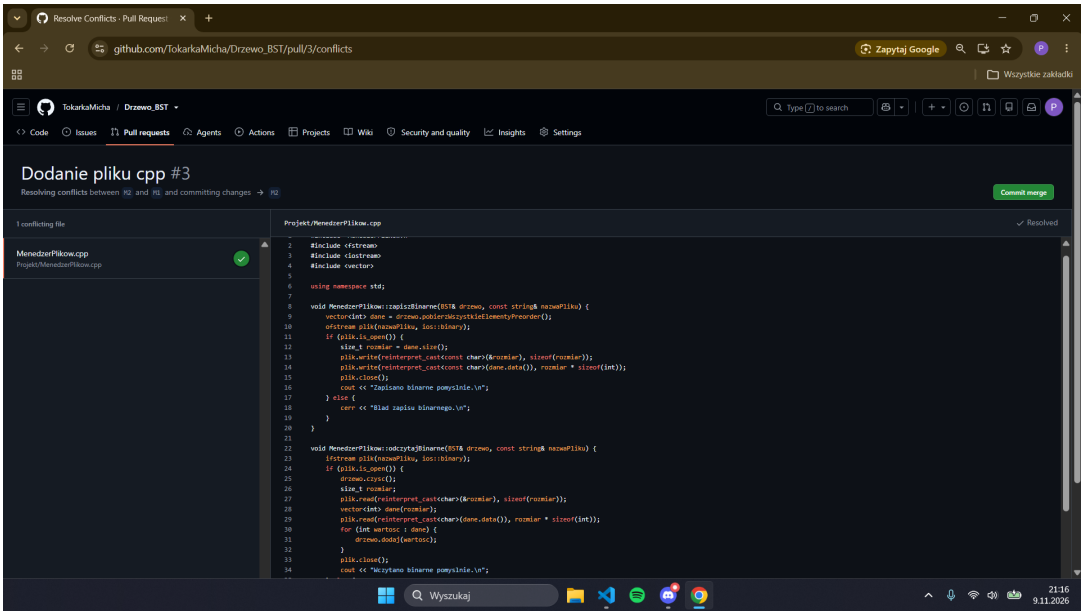
4.2. Zarządzanie konfliktami scalania (Merge Conflicts)

Integracja ciągła (ang. *Continuous Integration*) w zespołach deweloperskich jest krytycznie ważna. Na obrazie 4.2 przedstawiono proces, w którym operacja `git push origin M1` została siłowo odrzucona przez główny serwer GitHub (błąd *"non-fast-forward rejected"*). Sytuacja ta wynikła z faktu, że obaj partnerzy w zespole projektowym zmodyfikowali dokładnie ten sam plik (`main.cpp`), a serwer wymusił wykonanie `git pull` w celu ręcznego określenia, które poprawki są ważniejsze. Zjawisko to idealnie symuluje trudności napotykane w komercyjnych organizacjach informatycznych.

Rysunek 4.3 stanowi logiczne następstwo błędu z poprzedniego kroku. Po pobraniu logów (pull), zintegrowane środowisko Source Control edytora Visual Studio Code zatrzymało plik w stanie kwarantanny. Edytor udostępnia trzy warianty decyzji: *Accept Current Change* (zostawienie własnej pracy), *Accept Incoming Change*



Rys. 3.1: Wizualizacja usuwania węzła ”30”, posiadającego dwoje dzieci. W pierwszej kolejności algorytm poszukuje najmniejszego elementu w prawym poddrzewie (”35- tzw. następnik). Wartość następnika nadpisuje usuwany węzeł, a pierwotna pozycja następnika (zielona obwódka po lewej) zostaje zwolniona z pamięci.



Rys. 4.1: Środowisko deweloperskie obrazujące podgląd logów błędu kompilacji przy zapisie do pliku, wymuszającego korektę wskaźników (reinterpret_cast).

(zastąpienie pracy praca partnera z serwera) lub *Accept Both Changes* (połączenie ich). Precyzyjne rozwiązanie każdego z wystąpień gwarantowało, że aplikacja BST nie zgubi algorytmów drzewa pod natłokiem konfliktujących się wpisów.

4.3. Analiza grafu kontroli wersji na platformie GitHub

Weryfikacją całej ciężkiej procedury podziału środowiska, rozwiązywania konfliktów plików oraz tworzenia kolejnych udoskonaleń na osobnych gałęziach roboczych jest graficzna reprezentacja historii na GitHubie. Ostatni z udokumentowanych etapów prac (Rys. 4.4) przedstawia tzw. *Network Graph*.

Na osi czasu widać wyraźnie postępy prac prowadzonych niezależnie: fioletowa i zielona linia oznaczają boczne odgałęzienia projektowe `M1` i `M2`, w których zespół pisał kod i walczył z bugami bez wpływu na niebieską, chronioną gałąź centralną `main`. Zagięcia linii w górę, skierowane ze wskaźnikami `Merge pull request`, stanowią namacalne zatwierdzenie osiągniętych kamieni milowych przez współpracowników.

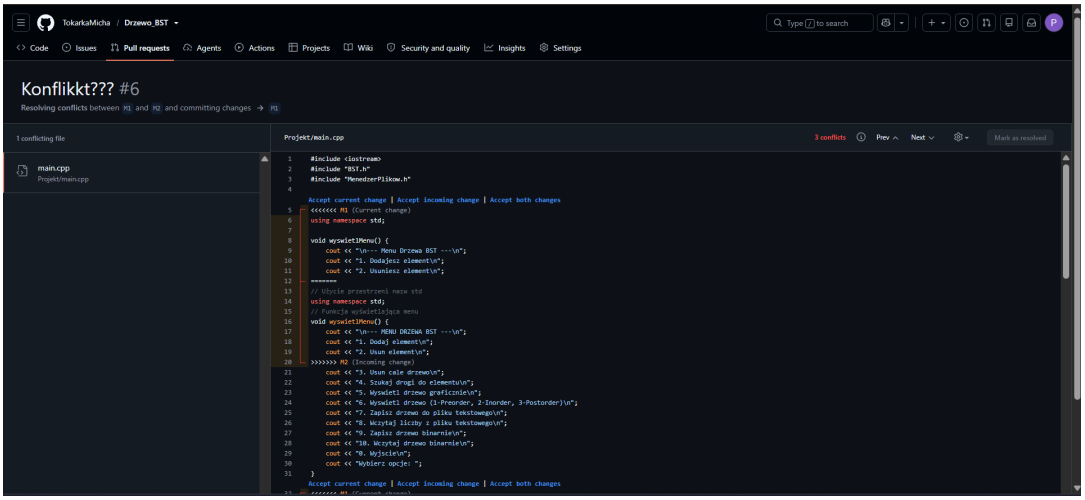
5. Rozległe podsumowanie, ewaluacja oraz perspektywy rozwoju projektu

Przeprowadzone w ramach niniejszego projektu prace inżynierskie udowadniają, że złożone algorytmy grafowe (jakim jest drzewo poszukiwań binarnych BST) wcale nie muszą stanowić archaicznego, akademickiego problemu. Umiejętne obudowanie ich nowoczesnymi wzorcami projektowymi języka C++, wydajnymi mechanizmami wejścia/wyjścia (I/O) oraz hermetyzacją, tworzy użyteczne i silne narzędzie aplikacyjne, które może posłużyć jako silnik wyszukiwania lub baza systemów decyzyjnych.

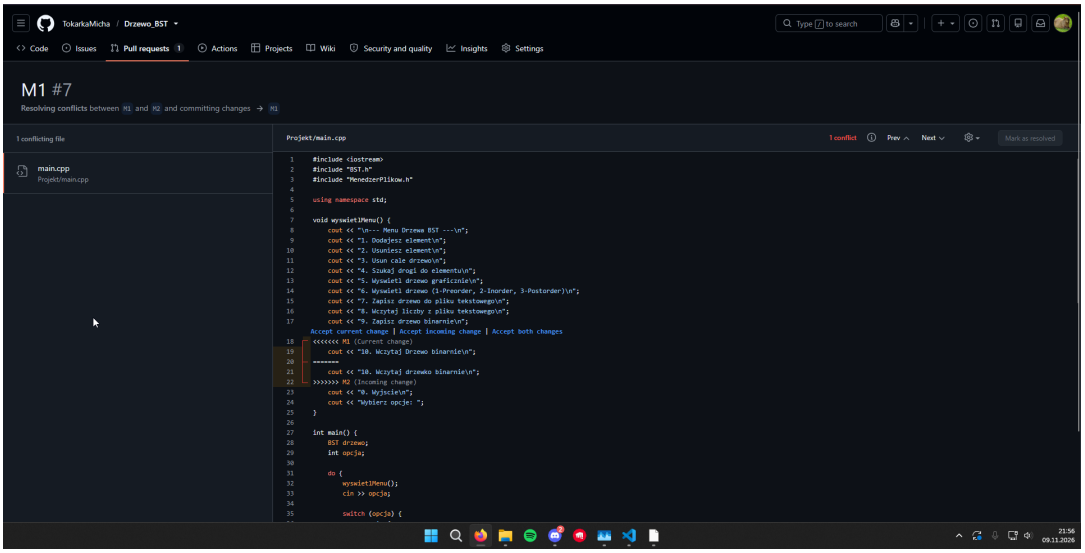
5.1. Ewaluacja założeń technologicznych i inżynieria błędu

Wdrożenie narzędzi Standardowej Biblioteki Szablonów drastycznie zmieniło perspektywę pracy nad surowym kodem. Wcześniejsze, naiwne operowanie dynamicznymi tablicami wskaźników przysparzało potężnych kłopotów z adresacją pamięci operacyjnej (zjawisko wielokrotnego niszczenia tej samej pamięci tzw. *double-free problem* lub odwoływania się do nieistniejących bloków *segmentation fault*). Wykorzystany z kolei w serializacji kontener `std::vector` samodzielnie ukrywa realokację pod warstwą bezpiecznych metod.

Praca z systemem kontroli wersji Git nauczyła rygorystycznego zarządzania re-



Rys. 4.2: Rozwiązywanie konfliktów w kodzie za pomocą github.



Rys. 4.3: Wbudowany system edycji konfliktów (Merge Editor) prezentujący sporną przestrzeń pliku `MenedzerPlikow.cpp`.



Rys. 4.4: Wizualizacja platformy GitHub przedstawiająca sieć utworzonych gałęzi deweloperskich, pomyślnie zrealizowanych scaleń i postępującego czasu zatwierdzeń (commitów).

wizjami. Udokumentowane konflikty (merge conflicts) to codzienność inżyniera ds. oprogramowania. Zrozumienie, w jaki sposób silnik Git różnicuje linie (diff) i dlaczego czasem odmawia automatycznej fuzji plików, przeniosło kulturę prowadzenia kodu zespołu na znacznie wyższy, profesjonalny poziom.

5.2. Krytyczna ocena rozwiązania i jego braki

Pomimo zaimplementowania pełnej gamy operacji dla konwencjonalnego układu BST, zaprezentowane rozwiązanie posiada wyraźne luki inżynieryjne z zakresu balistyki zasobów (Resource Management). Przede wszystkim, standardowe drzewo poszukiwań binarnych jest silnie podatne na niewłaściwy rodzaj danych wejściowych. Kiedy strumień z pliku podaje funkcjom wejściowym liczby posortowane kolejno od najmniejszej do największej (np. 1, 2, 3, 4, 5), tworzona struktura przypomina gałąź z prawymi dziećmi. Przestaje być drzewem, a cały wspaniały mechanizm podziału logicznego szukania z $O(\log n)$ drastycznie pogarsza parametry, osiągając koszmarną złożoność liniową $O(n)$. Zespół dostrzega ten mankament i identyfikuje to zjawisko jako tzw. "niezrównoważenie BST".

5.3. Proponowane ścieżki i perspektywy rozwoju (Future Works)

Mając świadomość zaimplementowanego rozwiązania, otwiera się pole do radykalnych usprawnień. Zespół wytyczył następujące wektory rozwoju dla ewentualnej wersji oprogramowania 2.0:

1. **Integracja struktury równoważącej:** Przekształcenie klasycznego modelu na model Drzewa Czerwono-Czarnego (ang. *Red-Black Tree*) lub modelu AVL, którego skomplikowane algorytmy rotacji węzłów na bieżąco korygowałyby wysokość drzewa bez względu na porządek liczb podawanych w wejściu dyskowym, gwarantując niezmienny czas wyszukiwania w najbardziej ekstremalnych bazach.

2. **Zaawansowane paradygmaty pamięci RAM (Smart Pointers):** Aktualny kod posiłkuje się "niezabezpieczonymi"wskaźnikami surowymi (`Wezel*`). Aktualizacja architektury języka przynajmniej do C++14 pozwoli na użycie inteligentnych wskaźników własnościowych `std::unique_ptr` oraz wskaźników współdzielonych `std::shared_ptr`. Środowisko C++ przejęłoby wtedy na siebie odpowiedzialność za mechanizm odśmiecania (Garbage Collection).

3. **Projekt graficznego środowiska i API:** Całkowita rezygnacja z przestarza-

łej konsoli powłoki na rzecz wykorzystania wysoce optymalizowanych silników interfejsu takich jak biblioteka Qt C++ lub lekka biblioteka Dear ImGui, a docelowo – wyabstrahowanie samego modułu matematycznego drzewa BST do formy skompilowanej Biblioteki Dołączanej Dynamicznie (pliki `.dll` lub `.so`), tak by inne systemy i mikroserwisy mogły korzystać z jej zasobów aplikacyjnych.

Bibliografia

[1] Scott Chacon and Ben Straub. *Pro Git - Oficjalny podręcznik systemu kontroli wersji*. Apress, 2014. Dostęp online: https://git-scm.com/book/en/v2.

[2] CppReference. C++ standard template library (STL) - dokumentacja kontenerów. https://en.cppreference.com/w/cpp/container, 2026.

[3] GeeksforGeeks. Binary search tree (bst) - data structure and algorithm tutorials. https://www.geeksforgeeks.org/binary-search-tree-data-structure/, 2026.

[4] GitHub Docs. Dokumentacja github - praca z repozytoriami i rozwiązywanie konfliktów. https://docs.github.com/en, 2026.